Migration vers une architecture de rendu multi-moteur

Brief de cadrage | Architecture multi-moteur, composants MCP et chantier terrain | Révision du 10 novembre 2026

Décision proposée

Organiser l'écosystème autour de cinq dépôts Git structurants, sans créer un dépôt par package NPM. Les modèles, données et algorithmes restent indépendants des moteurs 3D. Un monorepo de rendu publie les contrats graphiques, les kernels WebGL et WebGPU, puis les adaptateurs Babylon.js et Three.js. Le modèle physique de l'atmosphère appartient à celestial-core ; son rendu est une fonctionnalité générique intégrée par chaque adaptateur de moteur.

Objectif et périmètre

La migration rend les algorithmes de tuilage, de terrain et de rendu réutilisables dans plusieurs moteurs. Un seul runtime 3D Tiles assure la sélection, le raffinement, le streaming et le cycle de vie des contenus. Les pyramides DEM et imagerie Mercator ou cube sont exposées comme des Tilesets implicites. Les adaptateurs Babylon.js, Three.js ou autres traduisent les ports communs en objets et ressources propres au moteur.

Le brief couvre les dépôts, packages, dépendances, composants MCP et l'ordre de migration. Le chantier de raffinement morphologique et de surfaces paramétriques est détaillé en fin de document, avec socle nécessaire, extensions optionnelles et estimations conditionnelles. Le cadrage précède toute implémentation ; les noms et outils nouveaux restent des propositions.

Cible Git

Dépôt	Responsabilité	Statut / limite
celestial-core	Corps célestes, référentiels propres aux corps et modèle physique de l'atmosphère.	Dépôt existant. Pas de dépendance à un moteur graphique.
geodesy-ts	Transformations géodésiques, conversions de référentiels et projections.	Dépôt existant. Pas de rendu ni de dépendance Babylon/Three.
3d-tiles-core	Modèle 3D Tiles, runtime unique, disponibilités et streaming ; primitives de tuilage génériques dans @spacexr/tiles.	Dépôt existant. Ne contient pas la représentation spécialisée du terrain DEM.
terrain-core	Rasters, DEM, grilles CPU et sources terrain compatibles avec les Tilesets implicites du runtime commun.	Dépôt local initialisé, sans implémentation. Pas de ressources ni objets moteur.
spacexr-rendering	Contrats de rendu, algorithmes communs, kernels WebGL/WebGPU, adaptateurs moteurs et guides.	Nouveau monorepo NPM. Les packages internes peuvent être publiés séparément.

Quantia reste un dépôt transverse séparé, consommé par les projets qui en ont besoin. Il ne fait pas partie des cinq dépôts structurants de cette migration. Le total est donc de cinq dépôts principaux, ou six en comptant quantia.

Packages NPM visés

Conserver les identités NPM existantes, notamment @spacexr/geodesy, @spacexr/3d-tiles-core, @spacexr/3d-tiles-runtime, @spacexr/tiles, @spacexr/raster, @spacexr/dem, @spacexr/grid et @spacexr/3d-tiles-babylon. Les nouveaux noms restent des candidats jusqu'à la vérification des publications. Chaque composant peut être accompagné d'un package optionnel portant le suffixe -mcp.

Dépôt	Packages NPM candidats
celestial-core	@spacexr/celestial-core
geodesy-ts	@spacexr/geodesy
3d-tiles-core	@spacexr/3d-tiles-core ; @spacexr/3d-tiles-runtime ; @spacexr/tiles
terrain-core	@spacexr/raster ; @spacexr/dem ; @spacexr/grid ; autres surfaces seulement si nécessaires
spacexr-rendering	@spacexr/render-contracts · @spacexr/render-algorithms · @spacexr/render-kernels-webgl · @spacexr/render-kernels-webgpu · @spacexr/babylon-tiles3d · @spacexr/babylon-terrain · @spacexr/babylon-atmosphere · @spacexr/three-tiles3d · @spacexr/three-terrain · @spacexr/three-atmosphere

Ne publier un package que lorsqu'il possède une API stable et une frontière utile aux consommateurs. Les packages d'intégration moteur peuvent rester dans le monorepo spacexr-rendering tout en ayant des versions NPM indépendantes. Éviter les paquets vides créés uniquement pour compléter l'arborescence.

Frontières fonctionnelles

Tuilage et terrain

3d-tiles-core conserve le modèle, les codecs et le runtime unique, ainsi que @spacexr/tiles : adresses, pyramides génériques, voisinage et topologies Mercator/cube. terrain-core reçoit @spacexr/raster, @spacexr/dem et @spacexr/grid, les providers et les sources terrain compatibles avec les Tilesets implicites. ImplicitTilesetDecorator reste une passerelle du runtime commun. La représentation implicite peut être virtuelle, sans génération préalable de fichiers tileset et subtree. Les raccords entre faces du cube et les niveaux de détail utilisent la même traversée et les mêmes politiques de sélection. Aucun sélecteur terrain parallèle n'est créé.

Atmosphère

celestial-core porte la définition physique de l'atmosphère : composition, profil de densité, hauteur d'échelle et paramètres de diffusion. spacexr-rendering porte les algorithmes et les kernels de rendu. babylon-atmosphere et three-atmosphere relient ces calculs aux matériaux et cycles de rendu de leur moteur. Aucun dépôt atmosphere-core séparé n'est requis à ce stade.

WebGL et WebGPU

Les contrats et le comportement attendu des algorithmes sont communs. Les implémentations de kernels restent distinctes : GLSL pour le chemin WebGL, WGSL pour le chemin WebGPU. Les adaptateurs de moteur consomment le backend disponible et traduisent les ressources ou appels spécifiques au moteur. Ne pas promettre une source shader universelle si les APIs exigent des bindings ou des pipelines différents.

Dépendances autorisées

Projet	Peut dépendre de	Ne dépend pas de
celestial-core	Types mathématiques de base si nécessaire	Babylon.js, Three.js, WebGL, WebGPU
geodesy-ts	Types mathématiques de base	Moteurs 3D et modules de rendu
3d-tiles-core	geodesy-ts et primitives @spacexr/tiles hébergées dans ce dépôt	Babylon.js, Three.js, packages DEM/raster/grid de terrain-core
terrain-core	geodesy-ts, celestial-core selon besoin, @spacexr/tiles et contrats 3D Tiles/runtime	Moteurs graphiques, shaders et sélection concurrente du runtime
render-contracts / algorithms	Projets de domaine via interfaces stables	Babylon.js et Three.js
kernels WebGL / WebGPU	Contrats et algorithmes communs	Objets de scène Babylon.js ou Three.js
Adaptateurs moteur	Contrats, kernels, projets de domaine et moteur ciblé	Autres adaptateurs moteur

Plan de migration

1. Inventorier le code actuel. Classer chaque module, type, shader et dépendance selon sa responsabilité : 3D Tiles, terrain, algorithme graphique, WebGL/WebGPU ou moteur.
2. Établir les règles de dépendance et les contrats publics. Identifier les imports Babylon.js/Three.js présents dans les projets censés rester génériques, sans modifier encore le comportement.
3. Extraire terrain-core. Déplacer raster, DEM, grid, providers et métadonnées terrain, en conservant les identités NPM. Maintenir @spacexr/tiles et le runtime commun dans 3d-tiles-core ; préserver les passerelles existantes des pyramides vers les Tilesets implicites.
4. Créer les contrats de rendu et les algorithmes portables dans spacexr-rendering. Définir les entrées et sorties des kernels, les formats de ressources et la gestion des origines/coordonnées nécessaires au rendu planétaire.
5. Séparer les kernels par API graphique. Porter les algorithmes existants en GLSL et WGSL avec une sémantique documentée et des paramètres cohérents.
6. Migrer d'abord l'adaptateur Babylon.js existant et éprouver rapidement les mêmes contrats avec un chemin Three.js minimal. Terrain et contenus 3D passent par le même runtime. L'atmosphère suit les conventions spatiales communes et son propre cycle graphique.
7. Publier les packages, documenter les chemins de migration et maintenir des réexports temporaires depuis les anciens points d'entrée si des consommateurs en dépendent.

Critères de sortie

- Aucun import Babylon.js ou Three.js ne subsiste dans celestial-core, geodesy-ts, tiles3d-core, terrain-core ou les contrats/algorithmes génériques.
- Les pyramides DEM et imagerie Mercator et cube sont compatibles avec les Tilesets implicites et sélectionnées par le runtime 3D Tiles unique, sans obligation de générer préalablement leurs documents sur disque.
- Terrain et contenus 3D coexistent dans une scène avec la même implémentation du runtime ; plusieurs instances peuvent gérer des couches ou des corps distincts. Aucun runtime ni sélecteur terrain concurrent n'est introduit.
- Le modèle physique de l'atmosphère est défini une fois dans celestial-core et consommé par les intégrations Babylon.js et Three.js.
- Les kernels WebGL et WebGPU exposent des contrats cohérents, même lorsque leurs sources de shaders et leurs bindings diffèrent.
- Chaque package NPM documente son rôle, ses dépendances, ses moteurs requis et son chemin d'import public.

Décisions à garder ouvertes

Avant la première extraction, confirmer le scope NPM déjà utilisé et l'organisation des versions. Le choix de séparer les packages dans spacexr-rendering peut évoluer avec l'usage : garder plusieurs packages dans le monorepo, mais commencer avec un petit nombre de surfaces publiques. Les algorithmes, les contrats et les adaptateurs doivent rester séparés même si certains sont temporairement livrés ensemble.

Risques à piloter

Risque	Réponse de conception
Fuite d'API moteur dans les packages génériques	Garder les types moteurs dans les adaptateurs et faire passer les appels par des ports étroits.
Divergence entre WebGL et WebGPU	Documenter un comportement commun et maintenir les kernels GLSL et WGSL comme implémentations distinctes.
Confusion entre terrain et 3D Tiles	Conserver les données spécialisées du terrain et les exposer au runtime unique par une représentation en Tileset implicite.
Trop de packages NPM trop tôt	Garder les modules dans le monorepo, publier seulement les API stables et utiles.

Premier lot de migration

- Cartographier les imports, types et shaders existants dans tiles3d-core.
- Lister les consommateurs actuels et préserver leurs points d'entrée pendant la transition.
- Créer le squelette terrain-core et le monorepo spacexr-rendering avec leurs règles de dépendance.
- Déplacer un chemin terrain de bout en bout, puis valider la forme des contrats avant d'étendre l'extraction.

Composants MCP et behaviors

Chaque composant fonctionnel fournit une surface MCP optionnelle, distribuable sous la forme @spacexr/<package>-mcp. Cette surface s'appuie sur @cyanmycelium/mcp-core et expose les capacités réelles du composant à un agent. Le suffixe -mcp désigne un package compagnon ; le caractère @ reste réservé au scope NPM ou au séparateur de version dans une commande d'installation.

Séparation des responsabilités

Le package métier conserve ses API TypeScript et ses ports, sans dépendance MCP. Son package compagnon déclare les behaviors, schémas JSON, ressources, templates et grammaires. Un adaptateur MCP relie ces contrats à une instance existante du composant. Il appelle les mêmes API que l'application et ne réimplémente aucun calcul, chargement ou moteur de sélection.

Un behavior décrit les capacités d'une catégorie de composant. Les instances sont identifiées par des URI et des identifiants explicites. Introduire un behavior par composant fonctionnel pertinent, pas par classe, fichier ou objet de scène. Plusieurs behaviors peuvent être livrés dans un même package compagnon.

Le terme adaptateur MCP désigne le binding entre outils et API métier. Il est distinct de l'adaptateur de rendu Babylon ou Three. Les behaviors génériques ne manipulent aucun Scene, Mesh, Object3D, GPUDevice ou handle graphique sérialisé.

Dépendances et composition

Dépendances autorisées : package compagnon MCP vers package métier, ports publics et mcp-core. Le package métier n'importe jamais son compagnon. Les intégrations MCP Babylon et Three peuvent importer leur moteur ; les behaviors communs dépendent uniquement des ports de rendu indépendants du moteur.

Explorer ou une application hôte enregistre les behaviors autour de ses instances vivantes. Les packages compagnons ne démarrent ni serveur, ni transport, ni connexion par effet de bord. Le transport et la composition du serveur restent au niveau de l'hôte ; un serveur MCP est créé par session selon le modèle de mcp-core.

Le runtime 3D Tiles reste unique. Un RuntimeBehavior inspecte ou configure ce runtime ; TerrainBehavior et les behaviors de pyramide exposent leurs données et paramètres en utilisant ses ports. Aucune commande MCP ne crée un sélecteur terrain concurrent.

Packages MCP par composant

Noms candidats conservant le nom du package métier et ajoutant -mcp. Une ligne définit une responsabilité cible ; la publication intervient lorsque son API est utile et réellement implémentée. Les behaviors de composants internes peuvent d'abord rester regroupés.

Dépôt cible	Package compagnon candidat	Behavior et surface indicative
celestial-core	@spacexr/celestial-core-mcp	CelestialBehavior : catalogue, corps, repères, rotations et éphémérides. AtmosphereModelBehavior selon l'arbitrage du modèle physique.
geodesy_ts	@spacexr/geodesy-mcp	GeodesyBehavior : conversions, projections, ellipsoïdes et repères avec unités explicites.
3d-tiles-core	@spacexr/3d-tiles-core-mcp	TilesetBehavior : inspection et validation des documents et structures implicites.
3d-tiles-core	@spacexr/tiles-mcp	TilePyramidBehavior : adressage, faces, voisins, niveaux et disponibilité des sources.
3d-tiles-core	@spacexr/3d-tiles-runtime-mcp	RuntimeBehavior : état, sélection, budgets, chargements et diagnostics du runtime vivant.
terrain-core	@spacexr/raster-mcp	RasterBehavior : métadonnées, échantillonnage, résolution et références de données.
terrain-core	@spacexr/dem-mcp	DemBehavior : altitude, profil, pente et provenance, selon les fonctions disponibles.
terrain-core	@spacexr/grid-mcp	GridBehavior : topologie, jonctions, niveaux et diagnostics géométriques.
spacexr-rendering	@spacexr/3d-tiles-babylon-mcp	Binding Babylon des behaviors de rendu communs, diagnostics et capacités effectives.
spacexr-rendering	@spacexr/three-tiles3d-mcp	Binding Three des mêmes contrats de rendu ; nom aligné sur le package métier final.

Les autres composants de rendu suivent la même convention, par exemple @spacexr/babylon-terrain-mcp ou @spacexr/babylon-atmosphere-mcp si les packages métier correspondants sont effectivement créés. Les contrats, algorithmes et kernels internes sont décrits par les ressources de capacité et diagnostic ; un compagnon autonome n'est publié que s'il apporte un usage propre.

Les compagnons restent dans le dépôt du composant concerné, en workspace. Aucun dépôt MCP par package n'est nécessaire. Quantia reste hors des cinq dépôts structurants ; un éventuel quantia-mcp demeure optionnel et hors du premier lot.

Grammaires et contrats MCP

Référence technique locale : @cyanmycelium/mcp-core 1.4.1. La grammaire est une couche de descriptions et de formulations ; les schémas JSON définissent les arguments et résultats. Elle ne constitue ni un parseur de langage naturel, ni un moteur d'exécution, ni une politique d'autorisation.

Structure des behaviors

Les behaviors étendent McpBehavior et définissent _buildTools(), _buildResources(), _buildTemplate() et _buildGrammars() selon leur surface. Les bindings étendent McpAdapterBase ou implémentent IMcpBehaviorAdapter. L'exécution passe par executeToolAsync et la lecture par readResourceAsync. Les entrées Node restent dans l'hôte ou un point d'entrée /node ; aucun import /node dans les bundles navigateur.

Chaque behavior embarque les profils default:en et default:fr. Les variantes par client restent facultatives. Les fichiers utilisent la forme structurée McpGrammarData : server, tools, resources, templates et phrases. Les noms d'outils et les identifiants restent stables ; title, description et descriptions de propriétés sont localisables.

Résolution et validation

Ordre de priorité croissante : behavior, adaptateur, configuration statique de l'hôte, store de grammaires. La résolution de locale et de client utilise la politique de mcp-core ; les règles de repli sont testées. Une modification de grammaire ne change ni les types, ni les unités, ni le sens d'un outil.

Les grammaires sont validées avec McpGrammar.check. La règle withWordingRule("default:en") impose un seul emplacement pour chaque description : schéma inline ou grammaire. Le choix retenu ici est la grammaire pour les descriptions. Les phrases traduites conservent leurs clés et paramètres de substitution. Le chargement depuis le disque via loadGrammarDirectory est réservé au point d'entrée Node ; les behaviors navigateur embarquent leurs données.

Identités et résultats

Préfixer les noms pour éviter les collisions : tileset_validate, tiles_address, runtime_getState, dem_sampleElevation et render_getCapabilities sont des exemples de conventions, non un catalogue déjà implémenté. Les URI de composants peuvent suivre spacexr://components/{componentId}/instances/{instanceId}. Les objets d'une scène sont résolus dans la session cible, avec sessionId et instanceId explicites si nécessaire.

Les résultats déclarent un outputSchema lorsque pertinent et renvoient structuredContent avec une représentation textuelle compatible. Les erreurs opérationnelles sont explicites. Un échantillon terrain indique corps, repère, unités, datum vertical, résolution, source et absence éventuelle de données. Les identifiants de tuiles du runtime restent éphémères ; ils ne servent pas d'identité métier durable.

Les ressources exposent configuration, capacités, états et diagnostics bornés. Les rasters, géométries et captures volumineux sont référencés par URI ou artefact. Les handles moteur et GPU restent privés à leurs adaptateurs.

Intégration MCP et critères de sortie

Capacités et cycle de vie

getToolSupport annonce Full ou Partial lorsque l'exécution existe, avec les limites documentées. Planned et None restent masqués. Un profil de grammaire ne peut rendre une opération disponible. Le contrôle d'accès de la session reste distinct de ce mécanisme de capacité.

Les changements passent par _forwardResourceContentChanged(uri) et _forwardResourceChanged() ; mcp-core gère les notifications et abonnements de session. Les états modifiés sont invalidés avant relecture. La télémétrie de rendu est regroupée ou limitée en fréquence, sans notification à chaque frame. Les adaptateurs libèrent leurs abonnements en fin de vie.

Les ressources dépendant du demandeur ne sont pas placées dans le cache partagé de la ressource racine du behavior. Les métadonnées de IMcpRequestContext restent des entrées non fiables ; l'hôte les valide et applique ses autorisations avant toute mutation. Les accès directs et MCP partagent le même mécanisme de modification et de révision de l'état.

Les actions longues utilisent les opérations applicatives existantes : identifiant, progression, état terminal et annulation propagée aux API qui la supportent. accepted ne signifie pas completed. Le mcp-core local déclare tasks/* non pris en charge ; ne pas présenter ces opérations comme des tâches protocolaires MCP ni supposer une propagation AbortSignal fournie par executeToolAsync.

Composition avec Explorer

Explorer possède déjà ExplorerBehavior, ExplorerAdapter, des grammaires FR/EN et des outils par domaine. Conserver ses noms et URI publics ; déléguer progressivement les opérations aux companions. Chaque opération a un propriétaire unique. Le même outil n'est pas enregistré deux fois par Explorer et un composant. Le broker reste une dépendance de l'hôte, via mcp-broker-provider lorsque nécessaire.

Le premier chemin comprend inspection et validation du Tileset, lecture de la pyramide, observation de la sélection réelle du runtime, échantillonnage DEM et capacités de rendu. Il s'exécute sur les mêmes instances que l'application et doit produire les mêmes résultats métier par API directe et par MCP.

Validation avant publication

- Aucun import MCP dans les packages métier ; aucun import Babylon ou Three dans les behaviors génériques. Un utilisateur du package métier n'installe pas MCP par dépendance inverse.
- Chaque composant fonctionnel livré avec une surface MCP possède un behavior et une grammaire documentés ; seuls les outils implémentés sont annoncés. Les companions et outils des extensions optionnelles suivent leur propre lot et ne bloquent pas le socle. Les packages internes sans surface autonome restent regroupés.
- Tests des schémas, descriptions, repli FR/EN, collisions de noms, routage des URI et gestion des instances ; tests serveur-client avec LoopbackTransport.
- Tests de notifications, isolation de session, erreurs et libération des abonnements. Tests de progression et annulation pour les opérations effectivement prises en charge.
- Même sélection du runtime et mêmes résultats métier pour des entrées équivalentes, avec des tolérances définies. La comparaison Babylon/Three entre dans le lot multi-moteur dès que le second adaptateur est disponible ; sa réalisation complète ne bloque pas le socle de raffinement DEM sur Babylon existant.
- Vérification des bundles navigateur, des peerDependencies MCP/moteur et du contenu des archives NPM ; publication seulement après validation des versions compatibles.

Sources locales examinées : mcp-core/AGENTS.md et README.md (Grammar layer) ; src/mcp.behavior.ts ; src/mcp.grammar.ts ; src/interfaces/mcp.behavior.interfaces.ts ; spacexr-explorer/src/mcp.ts. Les noms de packages, behaviors et outils de cette extension sont des cibles de migration, sans implémentation ni publication effectuée.

Raffinement morphologique et surfaces paramétriques

Ce chantier applique l'architecture commune à deux besoins : conserver une restitution stable du DEM lorsque la subdivision dépasse sa résolution native, puis représenter des falaises par des grilles orientées. Le socle DEM est nécessaire au terrain cible. Les cartes de crêtes, grilles de falaises et reconstructions avancées constituent des extensions à décider séparément. Aucun nouveau dépôt ni second moteur de sélection n'est créé.

Socle nécessaire à la livraison terrain

- Séparer spatialLOD, elevationLOD et densité de grille. Le runtime 3D Tiles décide de la subdivision spatiale ; la pyramide indique le niveau et la zone de données réellement disponibles. La fin de la source DEM ne doit pas être confondue avec la fin de la hiérarchie spatiale autorisée.
- Échantillonner directement le DEM natif ou son ancêtre de référence, avec transformation UV explicite. Les descendants n'interpolent pas des rasters déjà interpolés. Documenter l'orientation des faces, les unités, le datum, les zones absentes et les conventions de déplacement.
- Conserver une altitude stable aux mêmes coordonnées, dans une tolérance fixée avant les tests. Rendre cohérents le déplacement, les normales et les requêtes CPU de mesure ou de picking ; mesurer et documenter les approximations de la géométrie de picking.
- Vérifier les frontières de tuiles et de faces cube, les coins et les transitions LOD. Conserver des volumes et erreurs géométriques prudents, y compris lors du raffinement et du changement de source. Utiliser les ports, caches et politiques d'activation existants.
- Établir une référence bilinéaire, instrumenter le chemin réellement exécuté et comparer à un éventuel chemin bicubique à filtrage et densité contrôlés. Le document source évoque du cubique ; le code examiné comporte aussi du bilinéaire. Aucun correctif d'interpolateur n'est présumé avant diagnostic.

Extensions optionnelles après validation du socle

Un bicubique contraint ou un mode edge-aware n'entre dans le lot que si la comparaison révèle un gain mesurable. Les cartes de crêtes multiscales et l'interpolation ridge-aware relèvent d'un lot morphologie distinct. Les falaises orientées suivent un prototype synthétique dédié, puis une qualification sur données mesurées. La découpe exacte, les supports courbes et la topologie adaptative sont des extensions ultérieures. L'annexe IA reste une hypothèse de recherche, hors engagement de livraison.

Limites de la garantie

Un LOD spatial plus fin ne crée aucune précision altimétrique nouvelle. L'invariance porte sur le champ de hauteur évalué aux mêmes coordonnées ; la silhouette d'un maillage fini peut évoluer avec ses sommets. Un DEM ne définit pas à lui seul une paroi masquée ou un surplomb. Un mode d'accentuation visuelle éventuel est explicite et séparé du mode fidèle aux données.

Répartition du chantier terrain par dépôt

Dépôt et composant	Nécessaire au socle	Optionnel et différé
3d-tiles-core / tiles	Ancêtre source, sous-zone UV, voisinage et topologie Mercator/cube. Réutiliser les primitives existantes.	Primitives communes supplémentaires seulement si un prototype de patches en démontre le besoin.
3d-tiles-core / runtime	Runtime unique : sélection, raffinement, disponibilité, cache, annulation et cycle de vie. Découpler disponibilité spatiale et disponibilité DEM.	Chargement et activation coordonnée des contenus de patch via les mêmes ports. Pas de gestionnaire LOD parallèle.
terrain-core / raster	Échantillons natifs, bilinéaire de référence, halos, enregistrement point/aire et données absentes.	Interpolateurs contraints ou anisotropes et rasters auxiliaires si leur gain est mesuré.
terrain-core / dem	Niveau source effectif, hauteur et dérivées cohérentes, provenance et résolution.	Description et calcul des caractéristiques de crête ; politique ridge-aware bornée.
terrain-core / grid	Grille réutilisable existante ; diagnostic de densité et raccords entre niveaux.	Contrats de surfaces orientées, supports, frontières, masques et coutures ; topologie adaptative plus tard.
spacexr-rendering	Référence CPU et kernels cohérents, déplacement et normales ; diagnostics dans Babylon WebGL/WebGPU existants.	Kernels de crêtes et patches, ressources auxiliaires ; portage Three qualifié dans le lot multi-moteur.
spacexr-tile-service	Réutiliser les données et métadonnées actuelles. Aucun nouveau traitement serveur obligatoire pour le socle.	Production, versionnement et distribution des cartes de crêtes, patches, validité et qualité.
geodesy_ts et celestial-core	Réutiliser ellipsoïdes, repères et conventions de hauteur existants.	Ajouter uniquement les transformations manquantes démontrées ; aucune logique de sélection ni shader.
spacexr-explorer	Scène de référence et observation des instances réelles du runtime et du terrain.	Scènes de falaises, comparaisons morphologiques et intégration des nouveaux contenus.

Contrats publics minimaux

Formaliser la référence source d'altitude : adresse stable de la donnée, niveau effectif, sous-zone et orientation, enregistrement raster, résolution, unités et données absentes. Garder les identifiants éphémères du runtime distincts des identifiants persistants des données. N'ajouter les références de crêtes ou de patch qu'avec leur lot ; le contrat SurfacePatch du document source reste non normatif.

Les behaviors et leurs bindings appartiennent aux packages compagnons du dépôt propriétaire. Le domaine conserve ses API sans dépendance MCP. La production de données et leur représentation GPU restent séparées.

Phasage et estimation du socle terrain

Estimation de cadrage, à confirmer après N0 : un développeur expérimenté à temps plein, dépôts et dépendances accessibles, runtime et scène Babylon de référence fonctionnels, jeux synthétiques disponibles. Les charges ci-dessous couvrent ce chantier terrain incrémental ; elles excluent la migration complète des dépôts, un adaptateur Three complet, les publications et les extensions optionnelles. Les accès aux données et délais de validation s'ajoutent au calendrier.

Lot	Résultat et condition de sortie	Charge indicative
N0 / Nécessaire	Figer source, caméra et géométrie ; relever interpolation, filtrage, UV et coûts. Définir les tolérances et reproduire les écarts éventuels.	2 à 3 j
N1 / Nécessaire	Consolider les contrats spatialLOD/elevationLOD, l'accès direct à l'ancêtre et la cohérence CPU/GPU. Tests unitaires synthétiques et cas d'overzoom.	4 à 7 j
N2 / Nécessaire	Intégrer et contrôler les transitions, normales, faces/coins cube, Mercator et bornes spatiales sur le runtime existant.	3 à 5 j
N3 / Nécessaire	Exposer un diagnostic MCP minimal de source, sélection et capacités en lecture, avec grammaires FR/EN et API existantes.	2 à 3 j
N4 / Nécessaire	Régressions WebGL/WebGPU sur Babylon, picking, budgets, documentation et validation des consommateurs touchés.	2 à 4 j
C1 / Conditionnel	Corriger une cause supplémentaire révélée par N0/N2 : précision UV, filtrage, couture ou chemin de rééchantillonnage. Rechiffrer si le runtime est structurellement affecté.	3 à 7 j supplémentaires

Lecture du calendrier

Le socle représente 13 à 22 jours de réalisation. Avec une réserve de 25 %, prévoir 17 à 28 jours ouvrés, soit environ 4 à 6 semaines à disponibilité complète. Si C1 est nécessaire, l'enveloppe devient 20 à 37 jours avec la même réserve, soit environ 4 à 8 semaines. Ce sont des fourchettes de planification, pas des dates promises. N0 confirme les hypothèses ; N2 déclenche un nouvel arbitrage si les mesures sortent de l'enveloppe.

Ordre recommandé : référence N0 avant déplacement des modules concernés ; extraction architecturale de ces modules ; N1 et N2 ; N3 et N4. Les travaux de migration sont chiffrés séparément et intercalés explicitement. Ne pas additionner implicitement ce calendrier au temps de migration ni supposer des semaines parallèles avec une seule personne.

Jalon de livraison

Le socle est livrable sans cartes de crêtes ni falaises : stabilité mesurée à source constante, absence de coutures au-delà des tolérances, bornes conservatrices, résultats CPU/GPU cohérents et coût compatible avec le budget relevé en N0. Les seuils de précision, mémoire et temps GPU sont fixés sur le matériel de référence avant qualification. Une régression bloque la livraison ; une amélioration esthétique optionnelle ne la bloque pas.

Extensions terrain et décisions de poursuite

Les lots suivants nécessitent une décision explicite de priorité et de budget après le socle. Chaque fourchette correspond à une expérimentation ou à un périmètre limité ; elle ne constitue pas un devis de généralisation. Ils ne sont pas cumulés dans les 4 à 8 semaines du socle.

Lot optionnel	Périmètre et décision de sortie	Charge indicative
O1 / Interpolation	Comparer bicubique contraint et/ou edge-aware à la référence. Conserver seulement si le gain dépasse le bruit de mesure et respecte les bornes.	4 à 8 j
O2 / Morphologie	Cartes de crêtes, halos et confiance ; premier kernel ridge-aware, mesures de déplacement et coût. Poursuivre si le gain est objectivé face à une référence indépendante.	10 à 20 j de prototype
O3 / Falaise synthétique	Paroi plane de 100 m, grille orientée, déplacement natif, masque simple et raccords haut/bas pendant les transitions. Décision après démonstration sans trous ni doublons.	10 à 20 j de prototype
O4 / Paroi mesurée	Petit jeu LiDAR couvrant réellement la paroi, conversion et validation sur points indépendants. Dépend de O3 et de la disponibilité de données adaptées.	10 à 20 j, hors acquisition
O5 / Raccords avancés	Découpe exacte, supports courbes et cas entre plusieurs patches. Étude séparée après O3/O4 ; complexité géométrique encore incertaine.	10 à 25 j pour un cas limité
O6 / Recherche	Topologie adaptative, généralisation multi-planétaire des patches et génération IA. Études conditionnées aux résultats précédents.	Non engagé ; à chiffrer après preuve

Contraintes des surfaces orientées

Les patches sont des contenus supplémentaires sélectionnés par le runtime unique. Le contrat terrain porte repère, surface support, déplacement, frontières, emprise, validité et provenance. Son encodage et son raccordement aux contenus 3D Tiles doivent être spécifiés ; ils ne sont pas déclarés standards par ce brief. Le chargement ne dépend pas du rechargement du DEM de chaque descendant.

L'adaptateur prépare le patch et son masque avant d'ouvrir le terrain ; leur activation est coordonnée via les ports du runtime. En cas de données manquantes ou d'échec, une surface de remplacement reste affichable. Les frontières imposent la continuité de position C0 ; une rupture de normales est conservée aux arêtes vives. Les raccords robustes entre LOD et contenus sont qualifiés avant toute généralisation.

Données et incertitude

La chaîne serveur des crêtes ou des patches n'est ajoutée qu'avec O2 ou O4. Les essais utilisent d'abord des données synthétiques reproductibles. Les surfaces mesurées, reconstruites et synthétiques sont distinguées. L'imagerie monoculaire ou un DEM seul ne prouvent pas la géométrie d'un surplomb ; l'annexe IA reste exploratoire et ne conditionne aucun jalon nécessaire.

Behaviors MCP du chantier terrain

Le socle ajoute une surface de diagnostic limitée et en lecture. Les commandes de modification et les outils propres aux extensions arrivent avec leurs capacités réelles. Les noms ci-dessous sont des candidats à aligner sur les noms publics d'Explorer, sans doublons. Aucun serveur ni runtime supplémentaire n'est créé.

Compagnon et behavior	Socle nécessaire	Extension optionnelle
@spacexr/tiles-mcp / TilePyramidBehavior	tiles_getSourceRegion : adresse demandée, ancêtre effectif, UV et orientation.	Diagnostics détaillés de voisinage et de données morphologiques.
@spacexr/dem-mcp / DemBehavior	dem_getSamplingState et dem_sampleElevation : mode effectif, hauteur, source, résolution et validité. Réutiliser un outil équivalent existant.	Comparaison d'interpolateurs, inspection des crêtes et choix du mode si la session autorise cette mutation.
@spacexr/3d-tiles-runtime-mcp / RuntimeBehavior	runtime_getState : sélection réelle et références des contenus. La disponibilité spatiale est distincte du niveau DEM servi.	États de préparation et activation coordonnée des patches.
@spacexr/grid-mcp / GridBehavior	Ressource décrivant la densité et la topologie existantes ; ne pas créer un outil uniquement pour compléter le catalogue.	grid_inspectPatch et grid_validateBoundaries après O3 ; paramètres de découpe après qualification.
Compagnon de rendu / RenderingBehavior	render_getCapabilities et ressource de diagnostics bornée : backend, interpolateur et métriques disponibles.	Affichages UV/normales et modes A/B ; diagnostic des kernels et textures de crêtes ou de patches.
Service de tuiles / behavior de production	Aucune nouvelle API de production requise pour N0 à N4.	Construction et suivi des artefacts en O2/O4, via un compagnon dédié si une API métier existe.

Contrats et contrôles

Les schémas fixent unités, corps, repère, source et niveau effectif ; les réponses indiquent les données absentes et les tolérances applicables. Les grammaires default:fr et default:en décrivent les mêmes capacités sans en modifier le sens. Full ou Partial reflètent le support réel ; Planned et None restent masqués. Les diagnostics ne transportent ni handles GPU ni rasters complets.

Les bindings utilisent les instances existantes et les mêmes API que l'application. Les tests Loopback vérifient routage, résultats, grammaires, erreurs et abonnements. Les notifications sont bornées en fréquence. Les mutations optionnelles passent par les droits et révisions de session ; les traitements longs utilisent les opérations applicatives existantes et leur annulation effective.

Référence documentaire : Ridge_Preserving_Terrain_Refinement_Foundation_v4.docx, sections 1 à 11 et annexe A. Ce brief reprend les besoins et les répartit par lot ; les décisions et priorités décrites dans la pièce source ne déclenchent aucune implémentation automatique. Le socle décrit ici est la cible nécessaire ; les extensions restent soumises aux jalons de poursuite.